CMPS402 - Machine Intelligence: Assignment 1: Searching

Campus Courier

Implement a planning model, five search algorithms, and a heuristic for a courier moving through a known campus map. Use the supplied terminal app and replay viewer to test your code and investigate how the algorithms behave. This assignment is classical search: no training or image recognition is required.

Assessment	Details
Total	**7 marks per student**
Team	**Pairs**; each member is assessed individually in the discussion
Deadline	**Saturday 31 October 2026 at 11:59 PM**
Upload	One Python-only ZIP per pair, containing the `submission/` folder
Discussion	Explanation of your implementation and application to unfamiliar examples

Late submissions will be penalized heavily. A submission up to two hours late receives a lower late penalty than a submission more than two hours late. The lower-penalty window ends on Sunday 1 November 2026 at 1:59 AM. This is not a penalty-free grace period. Start the upload early and check that it finishes successfully.

Plagiarism and cheating are prohibited. Work with your registered partner. Do not copy or exchange solutions with other teams, distribute your solution, or submit someone else's work as your own. Each member must understand the submitted implementation and explain its decisions.

1. World rules

The whole map is known. Movement is deterministic, orthogonal, and restricted to the rectangular map. Coordinates are zero-based `(row, column)`.

Symbol	Meaning
`S`	Exactly one start
`G`	Exactly one destination
`.`	Ordinary traversable tile
`w`	Expensive terrain: **cost 5 to enter**
`#`	Wall: never traversable
`K`	At most one required pickup/access pass; collected automatically on entry
`D`	Door: enter only after collecting the pass

Every other legal move costs **1**, including entry into `S`, `G`, `K`, and `D`. The pass is never consumed. With a `K` on the map, the goal requires reaching `G` **after collecting it**; passing through `G` earlier does not finish the task. Without

`K`, reaching `G` completes the task and doors cannot be unlocked. Completed goal states have no legal successors. Unreachable maps are valid inputs.

There are no diagonal moves, wraparound, random transitions, or wait actions. Illegal actions are omitted from the search model's successors.

Required action order

`submission/model.py` supplies this constant:

```
ACTIONS = ("UP", "RIGHT", "DOWN", "LEFT")
```

`get_actions(state)` must return legal actions **in that order**, skipping illegal moves without reordering the remaining actions. Do not return a set or alphabetically sort the actions. From the top-left corner of the following map, the order is `['RIGHT', 'DOWN']`:

```
SwG
...
```

The direct route has 2 moves and cost 6. The detour has 4 moves and cost 4. Search algorithms do not all optimize the same quantity.

2. Install uv and start the app

Extract the student ZIP into its own folder. In VS Code, open that folder and choose **Terminal > New Terminal**. Windows Terminal also works. Do not use the editor's Output pane or Debug Console. The terminal's current folder must contain `app.py` and `pyproject.toml`, not just the `submission/` folder.

Install uv if needed

First try `uv --version`. If the command is recognized, skip installation. Otherwise use one of the official installation methods below.

Windows PowerShell with WinGet:

```
winget install --id=astral-sh.uv -e
```

Windows alternative using the official installer:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

macOS or Linux using the official installer:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

The installer commands download and execute a script. Use the official address only; consult the official uv installation instructions if you want to inspect the script first or use another installation method.

Close and reopen your terminal after installing, then check:

```
uv --version
```

Set up this assignment

Run these commands from the extracted assignment folder:

```
uv sync --locked
uv run --locked app.py
```

uv uses the supplied lock file and Python 3.14. The first setup may download Python and the terminal packages, so internet access may be needed. The default installation uses Rich and Questionary; Gymnasium

and NumPy are **not required for the assessed assignment**. Do not edit the lock file or Python version.

Use **Up/Down arrows and Enter**. Select **Back** to return or **Exit** to quit. Ctrl+C cancels the current action. Long menus scroll with the selection. If arrow input is unsupported, use:

```
uv run --locked app.py --plain-menu
```

The fallback uses numbers, with `0` for Back/Exit. All assessed operations also have direct commands in Section 6.

Recommended workflow

1. Select **Check setup**, then **Watch worked examples**. The recorded demo works before you implement anything.

2. Implement the model in `submission/model.py`, then BFS and DFS in `submission/search.py`.

3. Use **Run public checks** as you work. From the results menu, re-run all checks, failed checks, or one selected check; you can also inspect a result's details.

4. Implement UCS, greedy, A*, and the heuristic. Use **Run my agent on a map** to inspect the recorded behavior.

5. Select **Compare all algorithms** and use the results to prepare for discussion.

6. Enter both members' names/IDs, build the Python-only ZIP, and submit it as described in Section 8.

SETUP OK checks your environment, not your solution. NOT IMPLEMENTED is normal in the untouched starter. No manual logging is required.

3. Implement the planning model

Implement `CampusProblem` in `submission/model.py`. Choose a hashable state representation that contains enough information to determine future legal moves and goal status.

Entry point	Required behavior
`__init__(specification)`	Keep the supplied `MapSpec` as `self.map`; already supplied
`get_initial_state()`	Return the initial planning state
`describe(state)`	Convert your state to `WorldState(row, column, has_key)`
`is_goal(state)`	Return a Boolean under the world rules
`get_actions(state)`	Return legal strings in **UP, RIGHT, DOWN, LEFT** order
`get_successor(state, action)`	Return the next state for a legal action
`get_cost(state, action)`	Return the positive cost of that move

Keep inheritance from the supplied `Problem` class. Its `initial` property and `successors(state)` adapter are provided conveniences, not additional tasks. `MapSpec` exposes `rows`, `height`, `width`, `start`, `goal`, and `pickup`; `pickup` is `None` when absent.

`describe` is the common interface between your chosen state encoding and the supplied tools. It must report the actual situation. You may use tuples, frozen objects, or another correct hashable representation; staff checks do not require one particular encoding.

The package contains a parser and finite input/output examples, not a complete world implementation. Public checks compare your model with those examples. Private checks use a separate staff model and unseen maps.

4. Implement five generic search algorithms

Complete these functions in `submission/search.py`:

```
BreadthFirstSearch(problem, initial_state)
DepthFirstSearch(problem, initial_state)
UniformCostSearch(problem, initial_state)
AStarSearch(problem, initial_state, heuristic)
BestFirstSearch(problem, initial_state, heuristic)
```

`BestFirstSearch` means **greedy best-first search**. Return a **list of action strings**, or `None` when no solution exists. An initial goal returns `[]` without expansion. Use the function's `initial_state` argument; do not replace it with the map's original start. The supplied `solve(...)` dispatcher connects your functions to the app and needs no changes.

Use `problem.is_goal`, `problem.get_actions`, `problem.get_successor`, and `problem.get_cost`. A heuristic passed to a search entry point has signature `heuristic(problem, state)`. Algorithms must also work on small graphs where states are not coordinates and actions are not compass directions. Do not inspect campus tiles inside generic search.

Choose and operate your own frontier containers, for example `list`, `collections.deque`, or `heapq`. No frontier implementation is supplied. Optional helpers `Node(state, cost=0, parent=None, action=None)` and `reconstruct(node)` are available. Sharing helper code between algorithms is allowed.

Automatic recording and limits

Use the provided problem normally and return your actions. The wrapper records expansions automatically; the runner records the route, moves, cost, cumulative route cost, and elapsed time. Do not add logging hooks, special frontier wrappers, or invented statistics. The viewer does not measure peak frontier size or record every frontier removal.

The expansion budget is **10,000 per run**. The default local timeout is **10 seconds per check/run**; the private grader uses **20 seconds per test group**. You can increase a local timeout with `--timeout SECONDS` when diagnosing a slow run; that does not change grading limits. Speed rankings or using the whole budget do not earn marks.

5. Design a portable heuristic

Complete `heuristic(problem, state)` in `submission/heuristic.py`. Get the situation through `problem.describe(state)`, and read map metadata through `problem.map`.

Do not access `state.row`, unpack/index `state`, or depend on your own model's extra attributes or state class. Staff tests evaluate your heuristic with a correct model whose internal state representation may differ from yours.

Return finite, nonnegative values that are zero at completed goals, admissible, and consistent on reachable transitions. Design a lower bound stronger than the supplied baseline. For this published case, the initial estimate must be **greater than 5 and at most 9**:

```
S....G
......
K.....
```

6. Test and understand your results

The terminal menu is the recommended interface. If you prefer commands, run them from the extracted assignment folder:

Goal	Command
Check setup	`uv run --locked app.py setup-check`
Launch menus	`uv run --locked app.py`
Run all public checks	`uv run --locked app.py check`
Re-run check 14 with details	`uv run --locked app.py check --case 14 --details`
Re-run model checks	`uv run --locked app.py check --case Model --details`
Create worked-example replay	`uv run --locked app.py demo`
Run BFS on weighted terrain	`uv run --locked app.py run --map maps/weighted_detour.txt --algorithm bfs`
Run A* with your heuristic	`uv run --locked app.py run --map maps/demo.txt --algorithm astar --heuristic student`
Generate comparisons	`uv run --locked app.py experiments`
Build the upload ZIP	`uv run --locked app.py submit`
Show command options	`uv run --locked app.py --help`

Algorithm names are `bfs`, `dfs`, `ucs`, `greedy`, and `astar`. Heuristic names are `baseline` and `student`; BFS/DFS/UCS do not need a heuristic choice. To keep a named direct-command run, add `--output output/my-run.json`. Otherwise the next direct run overwrites `output/run.html`; menu runs use descriptive names.

Public checks and re-running

There are **14 public checks**: four model checks; five algorithm-priority checks; BFS duplicate suppression; FIFO ties; initial goals/unreachable cycles; heuristic lower-bound/consistency; and heuristic portability through `describe`.

After **Run public checks**, use the results menu to **Re-run all checks**, **Re-run failed checks**, **Re-run one check**, or read a check's details. Selected re-runs update those results while preserving the other displayed results. If you edited related code, re-run all checks before submission: older passing results are not a fresh check of your changes. Direct-command `--case` runs only the selected check(s).

Verdict	Meaning and next step
PASS	Available local checks passed; continue testing and preparing for discussion
NOT IMPLEMENTED	Complete the named starter function
WRONG ANSWER	Compare expected/actual behavior and re-run the smallest failing case
TIME LIMIT	Worker exceeded its time limit; check loops, imports, and duplicates
RESOURCE LIMIT	Expansion budget exceeded; check repeated states/stale entries

Verdict	Meaning and next step
RUNTIME ERROR	Fix the reported syntax/import/type/index error first
NO SOLUTION	Valid for a bundled map certified unreachable by its reference facts
LOCAL ONLY	Custom map has no independent published answer; inspect your model as well

Use the diagnostic's reason, file/line, and next step. Public passes are not a final mark or proof of all model rules. Private tests check the same stated requirements on additional maps/graphs. They assess algorithms and heuristics with a correct staff model separately from your own model, to avoid cascading penalties.

Replay and output files

Open `output/demo.html`, `output/run.html`, or a menu run's HTML file in a browser. Choose a **Map**, choose an **Algorithm**, then **Play** or **Step**. **Solution route** follows the returned actions; **Search exploration** shows hypothetical expanded states, not the courier walking. Step narration explains position, pickup, and cumulative route cost. Purple indicates possession of the pass.

Use **Compare this map** and a row's **View** button to compare recorded algorithms. **Issues only** filters the comparison without resetting playback; **Next issue** opens a failed run and its explanation. Heuristic choices appear for informed algorithms. Replays are saved after execution, not streamed during search, and work without a server or account.

Output	Use
`output/checks.txt`	Complete readable check details
`output/run.html`, `output/runs/*.html`	Recorded single-run playback
`output/experiments.html`	Map-by-map comparison and playback
`output/experiments.txt`	Readable comparison and failure details
`output/experiments.csv`	Spreadsheet-friendly comparison
`output/submissions/`	ZIP to upload

The app also saves JSON internally for its replay and diagnostic tools; you do not need to edit or submit it. Local route replay uses **your model**. Bundled reference facts and model examples help find errors, but do not independently certify every transition. Custom maps are explicitly labeled LOCAL ONLY.

7. Rubric and discussion preparation

Part	Marks	Basis
Correct code, shared	**3.00**	Model 0.50; BFS 0.25; DFS 0.25; UCS 0.50; greedy 0.50; A* 0.50; heuristic 0.50
Explanation, individual	**2.00**	Explanation of code and implementation
Harder application, individual	**2.00**	Applying conceptual understanding
Total per student	**7.00**	**3 shared + 4 individual**

Code receives criterion-level partial credit, not an all-or-nothing score or an overall pass-percentage conversion. Equivalent valid optimal routes are accepted. Controlled micrographs deliberately test the stated priority/action/tie rules; larger maps are not graded by matching an exact reference path or expansion count.

Each member explains the submitted implementation and reasons through small unfamiliar examples during discussion. You may draw, trace, and demonstrate; polished speaking is not assessed. Harder questions apply taught principles, not unannounced algorithms or Python trivia. **No written report, prepared-answer file, or separate counterexample file is required.** Producing a CSV or attractive graphics earns no separate marks.

Compare all algorithms automatically runs 12 common maps with five algorithms and an additional A* run using your heuristic: **72 runs**. All common experiment maps are solvable. Use the results to discuss moves versus cost, duplicates, changed inventory, heuristic quality, and the limitations of runtime/expansion comparisons. Fewer expansions do not always imply less runtime. Discuss frontier memory qualitatively; no frontier-size measurement is required.

8. Submission steps

1. Finish `submission/model.py`, `submission/search.py`, and `submission/heuristic.py`. You may add Python helpers inside `submission/`; use relative imports. Do not modify supplied runtime files for submission.

2. In the app, choose **Enter member names / IDs**, or fill the literal `MEMBERS` list in `submission/student_info.py`. Check both names and IDs carefully.

3. Re-run all public checks. Inspect unresolved failures and test your own cases. Packaging is not a correctness check.

4. Choose **Build Python-only submission ZIP**, or run `uv run --locked app.py submit`.

5. Locate the generated file under `output/submissions/`. Its name includes both members' names and IDs, for example `Ahmed_Ali_1230137__Abdelrahman_Hassan_1230067.zip`.

6. Open the ZIP to verify that it contains **Python files under** `submission/` **only**, including `student_info.py`. Do not upload the entire project, virtual environment, maps, CSVs, JSON, replay HTML, or optional-extension files.

7. **One member uploads the pair's ZIP to the Google Classroom before the deadline.** Verify the pair's details, complete the submission action, and check the confirmation/status. Keep a local copy of the exact uploaded ZIP.

Do not wait until 11:59 PM to package or upload. The recorded successful submission time determines lateness. Late submissions are penalized as stated at the start of this document.

9. Optional Gymnasium extension

This extension is **optional, ungraded, and not a bonus**. It is separate from the public checks and is not part of the 7 marks. Attempt it only after completing the assessed work. Keep its files outside `submission/` and do not include them in your upload.

`optional_gym/environment.py` supplies a `CampusCourierEnv` starter: constructor, observation/action spaces, serialization through `describe`, action-mask helper, and text rendering. Complete `reset` **and** `step` as a thin adapter around your completed model. Do not duplicate the world rules. No reinforcement-learning algorithm is required.

Install the optional locked dependencies and run its separate checks:

```
uv sync --locked --extra gym
uv run --locked --extra gym -m optional_gym.checks
```

The terminal app also has **Optional Gymnasium extension**. That menu never adds these checks to required public-check results. Optional dependencies are selected using uv's documented extras mechanism.

Adapter contract

• Constructor: `CampusCourierEnv(specification, max_steps=100, render_mode=None, model_class=None)`. The default model class is your `CampusProblem`; staff can supply another compatible model.

• Action numbers: **0 = UP, 1 = RIGHT, 2 = DOWN, 3 = LEFT**. `action_space` is `Discrete(4)`.

• Observation: a dictionary with integer `row`, `column`, and `has_key` (0 or 1), obtained through your model's `describe`. The supplied observation space describes their valid ranges.

• `reset(seed=..., options=...)`: call `super().reset(seed=seed)`, restore the model's initial state, set `_steps = 0` and `_finished = False`, then return `(observation, info)` using the supplied helpers.

• `step(action)`: require reset and an unfinished episode; reject an out-of-range action with `ValueError`. For a legal move, use the model's cost and successor functions. For an in-range but illegal move, stay in place and use cost 1. Count every attempted in-range action toward `_steps`.

• Reward is **negative move cost**, with no goal bonus. Return `(observation, reward, terminated, truncated, info)`. `terminated` means the model's goal is satisfied. `truncated` means the step limit was reached without completion; if both occur on the same move, termination takes precedence. Set `_finished` when either is true; stepping before reset or after the episode ends raises `RuntimeError`.

• `info` contains the supplied `action_mask` in the stated action order and the step count. `render()` returns the supplied text map with `@` at the courier. Rendering is optional; no graphics dependencies are required.

The eight optional checks cover reset/spaces/order, weighted costs, illegal moves/time limits, goal-versus-limit precedence, pickup/doors/early destination, reset/render, invalid actions/episode order, and Gymnasium API compatibility. NOT IMPLEMENTED/errors are expected until you complete the adapter and core model. Read the Gymnasium Env API for the return signatures.

10. Troubleshooting

• **uv is not recognized:** reopen the terminal after installation; run `uv --version`. Use the official installation guide if PATH still needs attention.

• **No pyproject.toml found:** change to the extracted assignment's root folder, not its parent or `submission/`.

• **Arrow keys do not work:** use a real terminal or `--plain-menu`. Direct commands remain available.

• **Heuristic passes map runs but fails portability:** use `problem.describe(state)` instead of your model's internal state representation; re-run check 14.

• **Unexpected trace/order:** check **UP, RIGHT, DOWN, LEFT**, FIFO ties, BFS's generation-time goal test, and stale/duplicate handling.

• **Old replay still visible:** finish the run and reopen or refresh its generated HTML file. A replay is a saved recording.

• **Optional checks cannot import Gymnasium:** use `--extra gym` for both installation and running. The required search assignment does not need it.

• **You need setup or accessibility help:** contact any of the teaching assistants early, so we can resolve this as early as possible.